Runtime Contracts

Purpose

Every Runtime within Shio Runtime follows the same architectural contract.

A Runtime is not defined by the technology it uses.

It is defined by the responsibility it owns.

The Runtime Contract establishes the minimum architectural guarantees that every Runtime must satisfy.

These guarantees ensure that independently developed Runtime systems remain compatible with the overall architecture.

What Is a Runtime?

A Runtime is an independent subsystem responsible for one well-defined aspect of the digital character.

Every Runtime:

- owns one responsibility;
- owns its internal state;
- owns its resources;
- exposes a public contract;
- observes Character State;
- publishes Events;
- evolves independently.

A Runtime should be understandable in isolation.

Ownership

Every Runtime owns its own implementation.

Examples:

Speech Runtime owns:

- speech pipeline;
- TTS queue;
- ASR pipeline.

Avatar Runtime owns:

- animations;
- blinking;
- breathing;
- procedural motion.

Memory Layer owns:

- memory retrieval;
- memory storage;
- memory consolidation.

Action Runtime owns:

- Tool Registry;
- Permission Guard;
- execution.

Ownership must never overlap.

If two Runtime systems own the same responsibility, the architectural boundary is incorrect.

Resource Ownership

Every Runtime owns every resource that it creates.

Examples include:

- worker threads;
- timers;
- GPU resources;
- network connections;
- audio devices;
- caches.

No Runtime should initialize, modify or dispose resources belonging to another Runtime.

Ownership always implies lifecycle responsibility.

Public Contracts

Every Runtime exposes a small, stable public interface.

Other Runtime systems interact only through this public contract.

Internal implementation details remain private.

Public contracts should evolve slowly.

Internal implementations may evolve freely.

Character State

Every Runtime may observe Character State.

A Runtime may contribute to Character State only within its own responsibility.

Examples:

Speech Runtime updates:

- speech state;
- listening state;
- activity.

Experience Layer updates:

- emotional state.

Avatar Runtime does **not** update emotions.

It renders them.

This distinction is fundamental.

Event Publishing

Every Runtime may publish events.

Events communicate completed facts.

Examples:

```text id="rtc01" SpeechFinished

ToolCompleted

MemoryStored

AnimationFinished
```

````
Events should never describe intentions.

Intentions belong inside Runtime logic.

Events describe observable history.

---

# Runtime Independence

Runtime systems never depend on implementation details of another Runtime.

For example:

Speech Runtime should not know:

- how Avatar Runtime renders expressions;
- how Memory Layer stores memories;
- how Action Runtime executes tools.

It only depends on published contracts.

---

# Lifecycle Contract

Every Runtime participates in the same conceptual lifecycle.

```text id="rtc02"
Bootstrap

↓

Initialize

↓

Start

↓

Observe

↓

React

↓

Shutdown
````
````

```
↓

Dispose
```

This lifecycle provides architectural consistency across the entire Runtime.

Individual Runtime implementations may introduce additional internal states, but should preserve the conceptual lifecycle.

Communication Rules

Runtime systems communicate using only architectural mechanisms.

Allowed:

- Character State;
- Event Bus;
- Public Contracts.

Forbidden:

- direct access to internal Runtime state;
- shared mutable objects;
- hidden dependencies;
- platform-specific shortcuts.

Maintaining these communication rules preserves Runtime independence.

Failure Isolation

Failures should remain local whenever possible.

If one Runtime encounters an error, other Runtime systems should continue operating whenever their responsibilities permit.

For example:

A temporary TTS failure should not stop:

- Avatar Runtime;
- Memory Layer;
- Action Runtime;
- Event Bus.

Graceful degradation is preferred over complete system failure.

Technology Independence

The Runtime Contract never depends on implementation technology.

Whether a Runtime uses:

- Electron;
- Three.js;
- Python;
- Rust;
- SQLite;
- Groq;
- OpenAI;

is irrelevant to the architectural contract.

Responsibilities define Runtime.

Technologies implement Runtime.

Extensibility

New Runtime systems should naturally integrate into the existing architecture.

A new Runtime should require:

- its own responsibility;
- its own lifecycle;
- Character State integration;
- Event publishing;
- public contracts.

Existing Runtime systems should require little or no modification.

The architecture grows through extension rather than restructuring.

Architectural Guarantees

The Runtime Contract guarantees:

- independent development;
- replaceable implementations;
- clear ownership;
- predictable lifecycle;
- loose coupling;

- scalable architecture;
- future compatibility.

Every Runtime is expected to comply with this contract regardless of its internal complexity.

The Runtime Contract is therefore considered one of the fundamental architectural pillars of Shio Runtime.